Manual de trabalho no GitHub

APS – Arquitetura e Desenho de Software · Repositório `LucasRikelmi/aps-arquitetura` · versão de 07/11/2026

> **Em uma frase:** cada item do documento é uma *issue*; o autor escreve o item numa *branch* própria e abre um *Pull Request* (PR); o revisor aprova ou pede mudanças no PR; só depois da aprovação o texto entra na `main`. Tudo fica registrado e serve de evidência para a nota individual.

1. Quem é quem

Letra	Cor	Membro	Usuário GitHub	Autor de	Revisor de
A	vermelho	Grazieli	`grazidsm`	1.1, 1.5, 1.7, G7	G1, G6, 2.1, 2.4, 2.8
B	verde	Kamilly	`kamy-beningo`	G1, 1.2, 1.3, 1.4, 1.6, G5	G7, 2.2, 2.5, 2.6, 2.7
C	azul	André	`alancardete`	G2, 2.1, 2.2, 2.3, 2.8, G6	G8, 1.1, 1.5, 1.6, 1.7
D	amarelo	Lucas	`LucasRikelmi`	2.4, 2.5, 2.6, 2.7, G8	G5, 1.2, 1.3, 1.4, 2.3

Nas issues, a label **membro-X** (cor forte) indica o autor e a label **revisor-X** (mesma cor, tom claro) indica o revisor. Itens em rodízio (G3, G4) e de todos (G9) não têm revisor fixo: qualquer outro membro pode aprovar.

Regras que não podem ser quebradas

- **Autor ≠ revisor** no mesmo item. O GitHub não deixa o autor aprovar o próprio PR.
- **Ninguém escreve direto na `main`.** Todo texto entra por PR aprovado.
- **Cada autor edita apenas o arquivo do seu item** (`aps/secoes/<id>.md`). Precisa mudar algo de outro item? Comente na issue dele.
- **Uso de IA é registrado** com o prompt literal, a data e a ferramenta, no PR e na issue G5 (#5).
- **Prazos:** o 1.1 (pilha) fecha em 08/11 e destrava o resto. Revisão cruzada de 13 a 14/11; correções em 15/11; PDF fechado em 17/11.

2. Preparação (uma vez só)

1. Aceite o convite do repositório: no e-mail do GitHub ou em `github.com/notifications`.
2. Aceite o acesso ao quadro (Project), se tiver recebido o convite. Sem ele, você vê o quadro mas não move os cartões.
3. Abra o quadro: `github.com/users/LucasRikelmi/projects/1`. Filtre por *Autor* ou *Revisor* para ver só os seus itens.

4. (Opcional) Se for usar o terminal, instale o Git e o GitHub CLI e rode `gh auth login`. **Não é obrigatório:** tudo neste manual pode ser feito pelo navegador.

3. Vocabulário mínimo

Termo	O que é, aqui
Issue	A tarefa de um item (ex.: `#12 [1.3] Rede: DNS, TCP, TLS`). Tem escopo, prazo, autor e revisor.
Branch	Uma cópia de trabalho separada da `main`. Uma branch por item: `item/1-3-dns`.
Commit	Um "salvamento" com mensagem. Pode haver vários por item.
Pull Request (PR)	O pedido para levar sua branch para a `main`. É onde acontece a revisão.
Review	A avaliação do revisor: *Comment*, *Approve* ou *Request changes*.
Merge pull request	Leva todos os commits do PR para a `main`, preservando o histórico de cada pessoa (único método permitido no repositório).

4. Fluxo completo

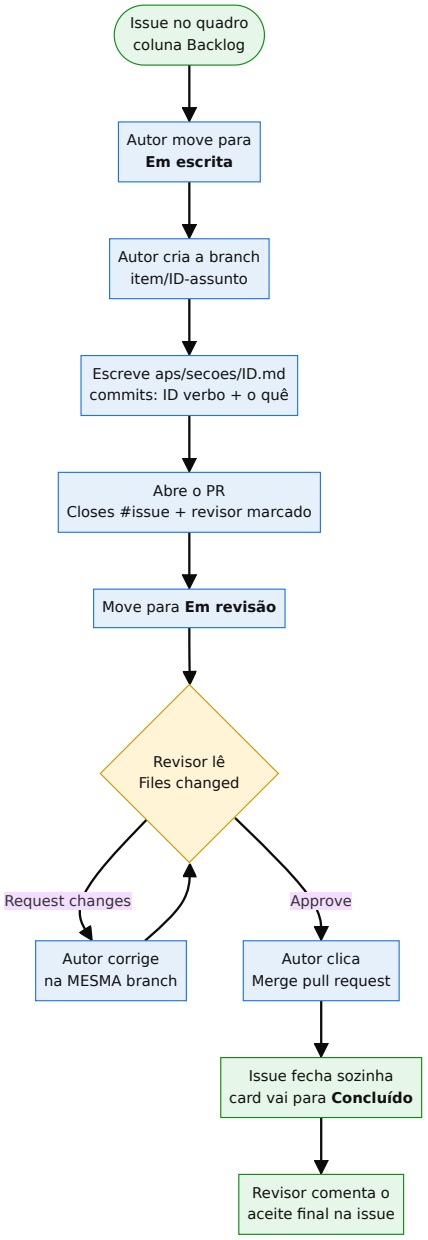


Figura 1 – Ciclo de um item. Azul: ações do autor. Amarelo: revisor. Verde: estados do quadro.

Coluna do quadro	Significa	Quem move
Backlog	Ainda não começou	–
Em escrita	Autor trabalhando na branch	Autor, ao começar
Em revisão	PR aberto, esperando o revisor	Autor, ao abrir o PR
Concluído	PR aprovado e mesclado	Automático, ao fechar a issue

5. Passo a passo do autor (pelo navegador)

Passo 1 – Pegar a issue. No quadro, abra o cartão do seu item e arraste para **Em escrita**. Leia o escopo e as dependências: se depender de um item que não está em *Concluído*, combine com o autor dele antes de começar.

Passo 2 – Criar a branch. Na página da issue, na barra da direita, em **Development**, clique em **Create a branch**. Troque o nome sugerido para o padrão `item/<id com hífen>-<assunto>` e confirme.

```
Exemplos: item/1-3-dns-tls   item/2-5-contrato-api   item/g5-registro-ia
```

Criar a branch pela issue já a deixa vinculada, o que facilita a rastreabilidade.

Passo 3 – Escrever. Na página do repositório, troque o seletor de branch (canto superior esquerdo, onde aparece `main`) para a sua branch. Entre em `aps/secoes/`:

- Se o arquivo ainda não existe: **Add file → Create new file**, com o nome `1.3.md` (o id do seu item).
- Se já existe: abra o arquivo e clique no ícone de lápis (**Edit**).

O ponto de partida é o trecho do seu item em `aps/documento/esboco-aps.md`. Figuras novas vão em `aps/diagramas/`.

Passo 4 – Commitar. Clique em **Commit changes…**, escreva a mensagem no padrão abaixo e deixe marcada a opção **Commit directly to the item/… branch**. Nunca escolha a `main`.

```
[1.3] descreve resolução DNS e handshake TCP
[1.3] adiciona handshake TLS 1.3 e certificado
[1.3] corrige termos apontados na revisão
```

Faça commits pequenos e frequentes: eles mostram a evolução do seu trabalho.

Passo 5 – Abrir o PR. Quando o texto estiver pronto para revisão, o GitHub mostra a faixa amarela **Compare & pull request**. Se ela não aparecer, use **Pull requests → New pull request** com base `main` e compare na sua branch. O formulário já vem com o template:

- **Título:** `[1.3] Rede: DNS, TCP, TLS`
- **Closes #12**, com o número da *sua* issue (tabela na seção 9). É isso que fecha a issue automaticamente no merge.
- Preencha o checklist do autor. Se usou IA, cole o prompt literal.
- Na direita, em **Reviewers**, selecione o seu revisor. Em **Assignees**, selecione você.

Depois mova o cartão para **Em revisão** e avise o revisor no Discord.

> Quer opinião antes de terminar? Abra o PR como **Draft** (seta ao lado de *Create pull request*). Quando estiver pronto, clique em **Ready for review**.

Passo 6 – Responder à revisão. Se o revisor pedir mudanças, edite o mesmo arquivo **na mesma branch** e commite. O PR se atualiza sozinho. Não abra um PR novo. Responda a cada comentário (por exemplo, "corrigido em abc123") e clique em **Resolve conversation**. Depois peça nova revisão no ícone de setas ao lado do nome do revisor.

Passo 7 – Mesclar. Com o PR aprovado, clique no botão verde **Merge pull request** e depois em **Confirm merge**. Os seus commits ficam no histórico da `main` como evidência da sua participação. Em seguida, clique em **Delete branch**. A issue fecha e vai para **Concluído**.

6. Passo a passo do revisor

Passo 1 – Abrir o PR. Você recebe uma notificação ("review requested"). Os seus pendentes estão em `github.com/pulls/review-requested`. Prazo de referência: até 24 h depois do pedido e, no máximo, até 14/11.

Passo 2 – Ler com o escopo ao lado. Abra a issue do item (link *Closes #...*) e a aba **Files changed** do PR. Verifique:

Critério	Pergunta
Escopo	Cobre tudo o que a issue pede? Inventou algo fora do escopo (o enunciado pune o não essencial)?
Correção técnica	As afirmações estão certas? Algum termo está usado de forma errada?
Consistência	Usa a mesma pilha definida no 1.1 e os mesmos nomes de componentes da Seção 2?
Figuras e tabelas	Estão numeradas, têm legenda e fonte, e são citadas no texto?
Referências	Toda afirmação não trivial tem fonte? A fonte aparece na lista (G6)?
IA	Se houve uso de IA, o prompt está registrado?
Arquivos	O PR mexe *apenas* no arquivo do item e nas figuras dele?

Passo 3 – Comentar. Passe o mouse sobre a linha e clique no **+** azul. Prefira comentários específicos e acionáveis. Para trocar um trecho, use o botão **Suggest changes**: o autor aceita com um clique.

```
Ruim: "melhorar esse parágrafo"
Bom:  "TLS 1.3 tem handshake de 1-RTT, não 2-RTT. Ver RFC 8446, seção 2."
```

Passo 4 – Decidir. Clique em **Review changes** (canto superior direito de *Files changed*) e escolha:

- **Approve**: está bom (pequenos detalhes podem ir como comentário).
- **Request changes**: há erro ou lacuna que precisa ser corrigido antes do merge.
- **Comment**: só dúvidas, sem decisão.

Um novo commit do autor **apaga a aprovação anterior**, então você precisa aprovar de novo. Isso é proposital: garante que você viu a versão final.

Passo 5 – Aceite final. Depois do merge, comente na issue: `Aceite: revisado e aprovado no PR #NN.` Esse comentário é a evidência pedida no Apêndice B.

7. Alternativa pelo terminal (opcional)

```
git clone https://github.com/LucasRikelmi/aps-arquitetura.git
cd aps-arquitetura
git switch main && git pull                 # sempre partir da main atualizada
git switch -c item/1-3-dns-tls              # cria a branch
# ... edita aps/secoes/1.3.md ...
git add aps/secoes/1.3.md
git commit -m "[1.3] descreve handshake TLS 1.3"
git push -u origin item/1-3-dns-tls
gh pr create --fill --reviewer kamy-beningo --body "Closes #12"   # ou pelo navegador
```

Para atualizar a branch com o que já entrou na main: `git switch item/1-3-dns-tls && git merge origin/main` (depois de `git fetch`). Não use `git push --force`.

8. Problemas comuns

Situação	O que fazer
Commitei na `main` por engano	Com a main protegida, o GitHub recusa o commit. Pelo navegador, escolha *Create a new branch* na tela de commit.
Esqueci o `Closes #`	Edite a descrição do PR e acrescente a linha. Vale enquanto o PR estiver aberto.
"This branch has conflicts"	Clique em **Resolve conflicts**, mantenha o texto certo, apague as marcas `<<<<<<<`, `=======`, `>>>>>>>` e clique em *Mark as resolved*. Se cada um editar só o próprio arquivo, isso quase não acontece.
Revisor não responde	Avise no Discord. Depois de 24 h, registre o atraso na issue (G4 acompanha).
Vou atrasar meu item	Comente na issue com a nova data e o motivo, **antes** do prazo. Atraso registrado conta melhor que atraso escondido.
Achei um erro no item de outra pessoa	Não edite o arquivo dela. Comente na issue ou no PR do item.
Usei IA	Cole no PR: ferramenta, data e prompt literal. Copie também para a issue #5 (G5).

9. Itens, issues e prazos

Issue	Item	Autor	Revisor	Prazo	Depende de
#1	G1 Infraestrutura da equipe	B	A	07/11	–
#2	G2 Pergunta formal à docente	C	qualquer	07/11	G1
#3	G3 Atas e registros	rodízio	qualquer	contínuo	G1
#4	G4 Monitoramento de compromissos	rodízio	qualquer	contínuo	G1
#5	G5 Registro de uso de IA	B	D	15/11	todos
#6	G6 Referências	C	A	15/11	1.x, 2.x
#7	G7 Integração e consistência	A	B	16/11	todos
#8	G8 Formatação e PDF final	D	C	17/11	G7
#9	G9 Apresentação	todos	qualquer	17/11	G7
#10	1.1 Visão geral e pilha	A	C	08/11	G2
#11	1.2 Browser	B	D	11/11	1.1
#12	1.3 Rede: DNS, TCP, TLS	B	D	11/11	1.1
#13	1.4 Servidor web	B	D	11/11	1.1
#14	1.5 Servidor de aplicação	A	C	11/11	1.1
#15	1.6 SGBD	B	C	11/11	1.1
#16	1.7 Ciclo completo	A	C	12/11	1.2–1.6
#17	2.1 Requisitos e escopo	C	A	09/11	G2
#18	2.2 Decisões arquiteturais	C	B	10/11	2.1
#19	2.3 Contexto	C	D	10/11	2.1
#20	2.4 Componentes	D	A	11/11	2.2
#21	2.5 Contrato da API	D	B	11/11	2.1
#22	2.6 Modelo de dados	D	B	11/11	2.2
#23	2.7 Desenho detalhado	D	B	12/11	2.4, 2.5, 2.6
#24	2.8 Segurança e erros	C	A	12/11	2.4–2.7

Exemplo real do fluxo: o PR #25 (*[G1] Sobe esboço, diagramas, WBS e template de issue*), revisado por A.